IMT Atlantique
Projet 28
Campus de Brest

UI AI Natives – Réinventer l'interaction homme-machine à travers les Generative UIs

Architecture du Projet

Version 1

Date 17/10/2026

Auteurs :
BADSSI Chaimae
BARET Alexis
BEDOUIN Arnaud
BELKHIRIA Zeineb
MOREAU Maxandre
SAFTA Yassine

Encadré par : STOENESCU Daniel

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1 Vue d'ensemble **2**

2 Organisation du dépôt **3**

3 Composants **4**

3.1 Frontend 4

3.2 Agent 4

3.3 Backend 6

3.4 Données 7

4 Contrats d'interface **7**

4.1 Page – runtime – agent : AG-UI 7

4.2 Agent – backend : outils MCP 7

4.3 Page – backend : API HTTP 8

4.4 Agent – page : catalogue A2UI 9

5 Transmission du jeton de session (à décider) **9**

6 Déroulement des cas d'usage **10**

6.1 Consultation et analyse 10

6.2 Virement 11

6.3 Produits et conseiller 11

1 Vue d'ensemble

L'application se compose de quatre programmes et d'une base de données hébergée sur Supabase (figure 1). Le navigateur n'exécute que la page ; les programmes tournent côté serveur, et la base est partagée par toute l'équipe.

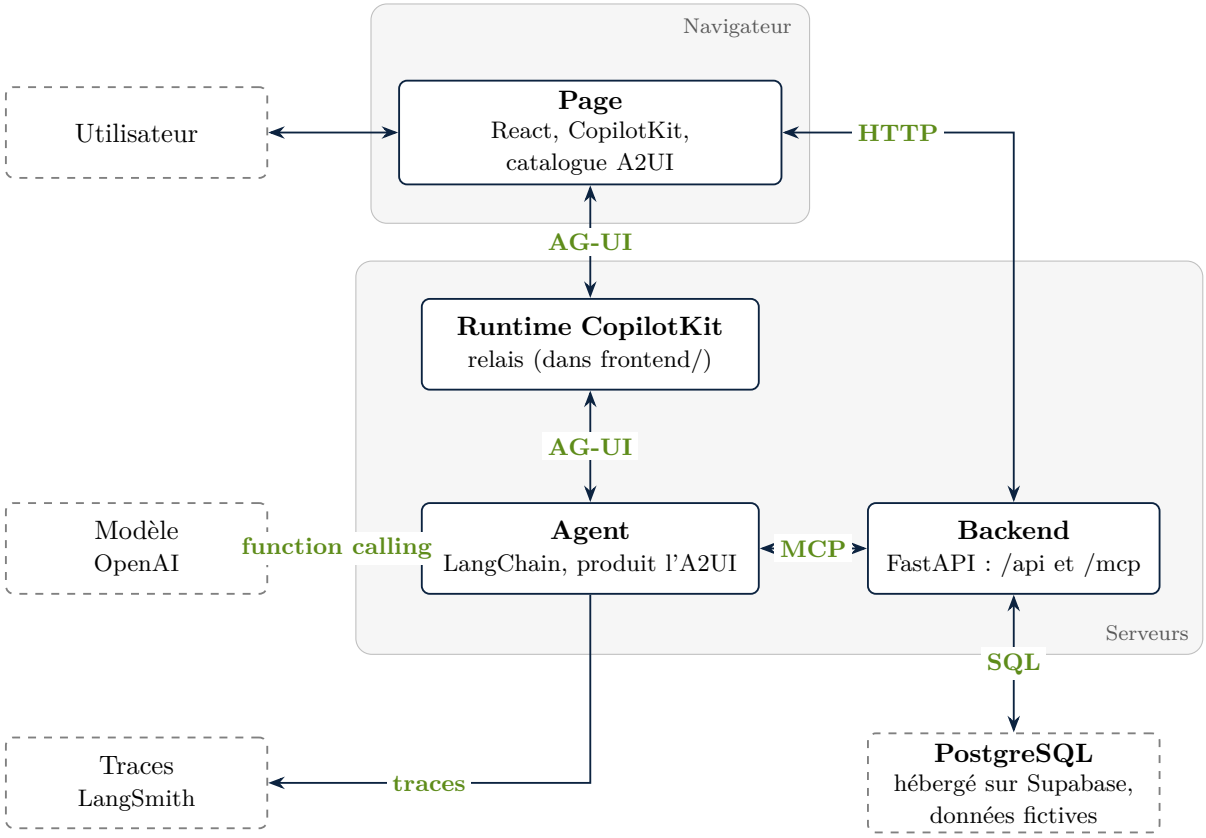


FIGURE 1 – Vue d'ensemble. Les cadres en tirets sont des services extérieurs. Le lien HTTP direct entre la page et le backend ne sert qu'à la connexion, dont le mode est à décider (section 2), et à la confirmation des virements.

Programme	Dossier	Port	Rôle
Page	`frontend/`	5173	Connexion (à décider), conversation, affichage des composants.
Runtime CopilotKit	`frontend/`	4000	Relais AG-UI entre la page et l'agent, traitement de l'A2UI.
Agent	`agents/agent_01/`	8001	Dialogue avec le modèle, appel des outils, description des interfaces.
Backend	`backend/`	8000	API bancaire (`/api`) et outils de l'agent (`/mcp`).
PostgreSQL	– (Supabase)	–	Base hébergée en ligne, partagée par l'équipe ; données fictives.

TABLE 1 – Programmes et ports en développement.

Quatre principes guident cette répartition :

1. **L'agent ne touche jamais la base de données.** Il n'agit que par les outils MCP du backend, qui appliquent les mêmes contrôles que l'API.
2. **Les règles métier sont écrites une seule fois**, dans la couche `services/` du backend, partagée par les routes `/api` et par les outils `/mcp`.
3. **L'identité vient de la page, jamais du modèle.** Le client concerné par une requête est déduit de la connexion de l'utilisateur, dont le mode est à décider (identifiant et mot de passe, ou profil type) ; aucun outil ne prend d'identifiant de client en paramètre.
4. **Le modèle ne peut pas exécuter de virement.** Il ne peut que préparer une simulation ; la confirmation passe directement de la page au backend, par une action de l'utilisateur (avec ou sans mot de passe selon le mode de connexion, à décider).

2 Organisation du dépôt

Les noms de fichiers, de dossiers, de routes, d'outils et de composants sont en anglais. Les éléments déjà présents dans le dépôt sont marqués « existant » ; les autres sont à créer.

```
generative-banking-ui/
|-- frontend/                    React + Vite + CopilotKit (existant)
|   |-- src/
|   |   |-- main.tsx             CopilotKitProvider (existant)
|   |   |-- App.tsx              root component, CopilotChat (existant)
|   |   |-- vite-env.d.ts        types of the VITE_ variables (existant)
|   |   |-- pages/               screens of the application (existant)
|   |   `-- catalog/             A2UI banking components (existant)
|   |-- runtime.ts               CopilotKit runtime (existant)
|   |-- index.html               (existant)
|   |-- vite.config.ts           (existant)
|   |-- package.json             (existant)
|   `-- .env.example             (existant)
|-- agents/                      one folder per agent
|   `-- agent_01/                banking assistant, LangChain (Python)
|       |-- main.py              FastAPI app + AG-UI endpoint (existant)
|       |-- config.py            environment variables (existant)
|       |-- agent.py             model, prompt, middlewares (existant)
|       |-- prompt.md            system prompt (existant)
|       |-- tools.py             MCP client to the backend (existant)
|       |-- ui.py                A2UI: page catalog, show_interface (existant)
|       |-- tests/test_ui.py     (existant)
|       `-- .env.example         (existant)
|-- backend/                     FastAPI + PostgreSQL (existant)
|   |-- app/
|   |   |-- main.py              mounts /api and /mcp (existant)
|   |   |-- api/                 auth.py, accounts.py, transactions.py,
|   |   |                        transfers.py, products.py, advisors.py
|   |   |                        (existant)
|   |   |-- mcp/server.py        agent tools (existant)
|   |   |-- services/            accounts.py, spending.py, transfers.py,
|   |   |                        products.py, advisors.py (existant)
|   |   |-- schemas/             request and response formats (existant)
|   |   |-- dependencies.py      database session, connected user
|   |   `-- security.py          login and session (to be decided)
|   |-- models/                  SQLAlchemy models (existant)
|   |-- migrations/              Alembic (existant)
|   |-- database.py              (existant)
|   |-- seed_database.py         (existant)
|   |-- tests/                   (existant)
|   `-- .env.example             (existant)
|-- database/diagrams/           UML diagram (existant)
```

```
|-- docs/                          état de l'art, architecture
|-- experiments/                   état de l'art experiments
|-- requirements.txt               Python dependencies, backend and agents (existant)
`-- README.md
```

Chaque dossier de premier niveau correspond à un chantier indépendant : il peut être confié à une partie de l'équipe sans gêner les autres, les échanges entre dossiers passant uniquement par les contrats de la section 4. Les dépendances Python du backend et des agents sont réunies dans un seul `requirements.txt`, à la racine, classées par partie ; celles de la page et du runtime sont dans `frontend/package.json`. Le dossier `agents/` contient un sous-dossier par agent : `agent_01` est l'assistant bancaire, et d'éventuels agents spécialisés, reliés par A2A, y prendraient place à côté.

Le contenu des dossiers `pages/` et `catalog/` de la page est laissé à la personne chargée du frontend. Les nouveaux modèles de `backend/models/` et leur migration sont laissés à la personne chargée du backend. Enfin, le mode de connexion n'est pas encore décidé (identifiant et mot de passe, ou simples profils types), ni par conséquent le besoin d'un jeton de session : `security.py`, la route `api/auth.py` et le trajet du jeton (section 5) en dépendent.

3 Composants

3.1 Frontend

La page est une application React construite avec Vite. Elle affiche la conversation CopilotKit ; le découpage en écrans (dossier `pages/`) est laissé à la personne chargée du frontend.

- **Connexion (à décider).** Deux modes sont envisagés. Avec un identifiant et un mot de passe, la page appellerait directement le backend (`POST /api/auth/login`) et joindrait le jeton de session reçu à chaque requête, vers le backend comme vers le runtime. Avec des profils types, l'utilisateur choisirait l'un des clients fictifs ; la page devrait tout de même indiquer au backend quel client est actif.
- **Conversation.** Le fournisseur `CopilotKitProvider` relie la page au runtime ; le composant `CopilotChat` affiche les messages, les étapes de l'agent et les surfaces A2UI.
- **Catalogue A2UI.** Le dossier `catalog/` contient les composants bancaires que l'agent peut afficher (section 4.4) ; ce sont des composants React ordinaires, au style de l'application. Un composant absent du catalogue ne peut pas être affiché.
- **Confirmation des virements.** Le composant `TransferConfirmation` demande la confirmation de l'utilisateur (et son mot de passe si la connexion en utilise un, à décider) et appelle directement le backend, sans passer par l'agent (section 6.2).

Le **runtime CopilotKit** est un petit serveur Node (`runtime.ts`, lancé par `npm run runtime`). Il déclare l'agent par son adresse AG-UI, lue dans la variable `AGENT_URL`, active le traitement de l'A2UI et transmet à l'agent l'en-tête `Authorization` reçu de la page, sans configuration particulière, ce qui servira si un jeton de session est retenu (section 5). Il partage le `package.json` de la page, ce qui garantit que les paquets CopilotKit de la page et du runtime ont la même version ; `@ag-ui/client` doit aussi rester à la version exacte qu'utilise CopilotKit.

3.2 Agent

L'agent (`agents/agent_01/`) est écrit en Python avec LangChain et exposé en AG-UI par un serveur FastAPI, comme dans l'exemple de l'état de l'art. La boucle de l'agent (appel du modèle,

exécution des outils demandés, nouvel appel du modèle, jusqu'à la réponse finale) n'est pas écrite à la main : elle est fournie par `create_agent`, que l'on se contente de configurer.

Fichier	Rôle
`config.py`	Lit les variables d'environnement, en un seul endroit.
`agent.py`	Construit l'agent : modèle, consigne, outils, middlewares et mémoire des conversations.
`prompt.md`	Consigne du modèle.
`tools.py`	Récupère les outils du serveur MCP du backend et leur ajoute l'identité de l'utilisateur (jeton de session, à décider).
`ui.py`	Permet à l'agent de décrire des interfaces en A2UI avec le catalogue de la page.
`main.py`	Serveur AG-UI (port 8001) : rend l'agent joignable par le runtime.

Table 2 – Fichiers de l'agent.

— **Modèle.** `gpt-6-luna`, fourni par la passerelle d'IA de l'IMT, compatible avec l'API d'OpenAI. Il est créé avec `ChatOpenAI` ; la clé d'abonnement est transmise à la fois comme clé d'API et comme paramètre d'URL `subscription-key`, comme l'exige la passerelle. Le nom du modèle et l'adresse sont des variables d'environnement.

— **Consigne** (`prompt.md`). Rôle d'assistant bancaire, interdiction d'inventer des données, obligation de passer par les outils et d'afficher les résultats avec les composants du catalogue plutôt qu'en texte.

— **Outils** (`tools.py`). L'agent ne définit pas d'outil métier : ils sont écrits dans le backend, à côté des données. Au démarrage, l'agent se connecte au serveur MCP du backend et en découvre la liste (`langchain-mcp-adapters`) ; un nouvel outil ajouté au backend est donc disponible au redémarrage suivant, sans modifier l'agent. L'identité de l'utilisateur (un jeton de session, si ce mode est retenu) est ajoutée à chaque appel d'outil (section 5).

— **Interfaces** (`ui.py`). L'agent n'affiche rien : il produit une description de l'interface en A2UI (du JSON), que l'événement AG-UI transporte jusqu'à la page, qui la rend avec ses composants React. Le mécanisme, mis au point dans l'essai `experiments/architecture/`, comporte trois éléments :

  — `page_catalog` lit le catalogue déclaré par la page, que le runtime place dans l'état de l'agent (`state["ag-ui"]["a2ui_schema"]`) ;

  — `catalog_prompt`, un middleware `dynamic_prompt`, ajoute le schéma de ces composants à la consigne de chaque appel du modèle, pour qu'il n'utilise que des composants existants ;

  — `show_interface` est le seul outil défini dans l'agent : le modèle lui passe les composants à afficher ; l'outil les valide (catalogue, racine, propriétés obligatoires) et renvoie soit le message A2UI, soit les erreurs, que le modèle corrige.

  Ce mécanisme a été vérifié avec un modèle Gemini ; il reste à le vérifier avec `gpt-6-luna`.

— **Mémoire des conversations.** L'historique de chaque conversation est gardé en mémoire (`InMemorySaver`) ; il est perdu au redémarrage de l'agent, ce qui suffit pour le prototype.

— **Point d'entrée AG-UI** (`main.py`). Il est écrit dans le projet plutôt que fourni par `add_langgraph_fastapi_endpoint`, car cette fonction ne transmet pas les en-têtes de la requête à l'agent, et l'identité de l'utilisateur, transmise en en-tête, serait perdue (section 5).

— **Observabilité.** Les traces sont envoyées à LangSmith (offre gratuite, compte unique de l'équipe), ce qui ne pose pas de problème avec des données fictives. Le jeton de session, s'il est retenu, n'y figure pas (section 5).

3.3 Backend

Le backend existant (FastAPI, SQLAlchemy, Alembic) devient le point d'entrée unique vers les données. Son application `app/main.py` monte deux parties :

— `/api` : les routes HTTP utilisées par la page (connexion selon le mode retenu, confirmation de virement) et utiles aux tests ;
— `/mcp` : le serveur MCP (`app/mcp/server.py`), en transport HTTP, qui expose les outils de l'agent.

Les deux appellent la même couche `services/`, qui contient la logique métier (calcul des dépenses par catégorie, préparation d'un virement…) et vérifie systématiquement que les données demandées appartiennent au client connecté. Le contenu du module `security.py` dépend du mode de connexion, à décider : avec un identifiant et un mot de passe, il gérerait le hachage et la vérification des mots de passe, le blocage temporaire après plusieurs échecs et les jetons de session. Le module `dependencies.py` ouvre la session de base de données et retrouve l'utilisateur connecté à partir de la requête, en un seul endroit pour `/api` et `/mcp`.

Dossier	**Rôle**
`api/`	Routes HTTP : elles reçoivent la session de base et l'utilisateur connecté, puis appellent un service. Aucune logique bancaire.
`mcp/`	Outils de l'agent : même rôle que les routes, avec l'identité de l'utilisateur lue dans l'appel MCP.
`services/`	Une fonction par opération, qui reçoit la session de base et l'utilisateur connecté et ne touche qu'aux données de ce client.
`schemas/`	Formats Pydantic des réponses : ce qui sort du backend est choisi (IBAN masqué, aucun champ interne) au lieu de renvoyer les modèles SQLAlchemy.
`tests/`	Règles du document : appels d'outils sans utilisateur refusés, données d'un autre client refusées, virement en deux temps, connexion (à décider).

Table 3 – Organisation du backend.

Le virement est découpé en deux fonctions de `services/transfers.py` : `prepare_transfer`, appelée par l'outil, crée la simulation sans rien débiter ; `confirm_transfer`, appelée uniquement par la route de confirmation, l'exécute ; une vérification du mot de passe à cette étape dépend du mode de connexion, à décider. Aucun outil n'appelle la seconde. La fonction qui écrit dans le journal d'audit reste à placer (`security.py` ou un service dédié).

Le serveur MCP est écrit avec `FastMCP`, du SDK officiel `mcp` (version 1.30.0), et monté dans l'application FastAPI. Un essai a permis de fixer les trois points suivants :

— le serveur est déclaré avec `streamable_http_path="/"`, puisque FastAPI le monte déjà sous `/mcp`, et avec `stateless_http=True`, chaque requête apportant sa propre identité d'utilisateur ;
— une application `FastMCP` montée dans FastAPI ne démarre pas seule son gestionnaire de sessions : le *lifespan* de `app/main.py` doit exécuter `mcp.session_manager.run()` ;
— un outil reçoit un paramètre `ctx` (`Context`), par lequel il lit les en-têtes de la requête HTTP (`ctx.request_context.request.headers`) ; ce paramètre n'apparaît pas dans le schéma de l'outil envoyé au modèle.

Dans l'essai, la liste des outils a été obtenue sans en-tête, et un outil a lu l'en-tête `Authorization` envoyé par un client MCP. L'adresse `/mcp`, sans barre finale, est redirigée vers `/mcp/` (code 307)

et l'appel aboutit : `BACKEND_MCP_URL` peut rester inchangée. Les variables de connexion à la base, lues par `database.py`, sont listées dans `backend/.env.example`.

3.4 Données

La base PostgreSQL est hébergée sur Supabase et déjà remplie avec les données fictives. Elle est partagée par toute l'équipe : une migration Alembic modifie les tables pour tout le monde, et n'est donc lancée que par la personne chargée du backend, après relecture. Supabase n'est utilisé que comme hébergeur de PostgreSQL ; le backend y accède avec SQLAlchemy, comme à n'importe quelle base PostgreSQL.

Les modèles existants couvrent les clients, conseillers, utilisateurs, comptes, transactions, catégories, cartes, produits et souscriptions. Le modèle `User` stocke aujourd'hui un code PIN haché (`app_pin_hash`, avec un compteur d'échecs et une date de blocage) : si la connexion se fait par mot de passe (à décider), ce champ est à remplacer par un mot de passe haché, par une migration Alembic. Quatre modèles prévus par le diagramme UML restent à créer :

Modèle	Fichier	Usage dans l'architecture
`UserSession`	`user_sessions.py`	Jetons de session émis à la connexion (empreinte du jeton, expiration, révocation), seulement si ce mode est retenu.
`AuditLog`	`audit_logs.py`	Journal des actions : connexions, appels d'outils, confirmations de virement.
`Beneficiary`	`beneficiaries.py`	Bénéficiaires enregistrés par le client, seuls destinataires possibles d'un virement.
`TransferSimulation`	`transfer_simulations.py`	Virement préparé par l'agent, en attente de confirmation (statut, date de confirmation).

TABLE 4 – Modèles à ajouter dans `backend/models/`.

4 Contrats d'interface

4.1 Page – runtime – agent : AG-UI

La page et l'agent échangent des événements AG-UI, relayés par le runtime : message de l'utilisateur et historique dans un sens ; texte en streaming, étapes, appels d'outils et surfaces A2UI dans l'autre. L'identité de l'utilisateur accompagne chaque requête dans l'en-tête `Authorization`, sous la forme d'un jeton de session si ce mode est retenu (section 5).

4.2 Agent – backend : outils MCP

Les outils sont déclarés dans `backend/app/mcp/server.py`. Aucun ne prend d'identifiant de client : le backend le déduit de l'identité de l'utilisateur transmise par l'agent (jeton de session ou profil type, à décider). Deux règles en découlent pour le serveur MCP :

— la **liste** des outils est accessible sans utilisateur, car l'agent la charge au démarrage, avant toute connexion ; l'**appel** d'un outil, lui, exige un utilisateur identifié ;
— un outil ne renvoie jamais le jeton de session, s'il y en a un, dans son résultat : ce résultat est transmis au modèle, à la page et aux traces.

Outil	Paramètres	Renvoie	Accès
`list_accounts`	–	Comptes du client, avec solde	Lecture
`get_recent_transactions`	`account_id`, `limit`	Dernières opérations d'un compte	Lecture
`get_spending_by_category`	`start_date`, `end_date`	Total des dépenses par catégorie	Lecture
`compare_spending`	début et fin de chacune des deux périodes	Dépenses des deux périodes et écart, par catégorie	Lecture
`list_beneficiaries`	–	Bénéficiaires enregistrés	Lecture
`prepare_transfer`	`source_account_id`, `beneficiary_id`, `amount`, `reason`	Simulation en attente et son récapitulatif	Écriture (simulation)
`list_products`	`product_type` (facultatif)	Produits bancaires disponibles	Lecture
`get_product_details`	`product_id`	Description et tarif d'un produit	Lecture
`get_advisor`	–	Conseiller attribué au client	Lecture
`get_appointment_slots`	`reason`	Créneaux de rendez-vous simulés	Lecture

Table 5 – Outils MCP exposés à l'agent.

Les comptes, bénéficiaires et produits sont désignés par leur identifiant, que le modèle obtient d'abord avec l'outil `list_*` correspondant. La description de chaque outil, que le modèle lit pour choisir, est la docstring de sa fonction dans `server.py`.

4.3 Page – backend : API HTTP

La page n'appelle le backend directement que pour deux opérations, qui ne doivent pas dépendre du modèle :

Route	Rôle
`POST /api/auth/login`	À décider : vérifie l'identifiant et le mot de passe, renvoie un jeton de session.
`POST /api/auth/logout`	À décider : révoque le jeton.
`POST /api/transfers/{id}/confirm`	Confirme la simulation si elle appartient au client et est encore en attente (après vérification du mot de passe, si la connexion en utilise un).
`POST /api/transfers/{id}/cancel`	Annule la simulation.

Table 6 – Routes appelées directement par la page. Les routes `auth` dépendent du mode de connexion, à décider.

Les autres données (comptes, transactions, produits) sont aussi exposées en lecture sous `/api`, en appelant les mêmes services que les outils MCP : elles servent aux tests et pourront servir à des écrans classiques.

4.4 Agent – page : catalogue A2UI

Composant	Données	Cas d'usage
`AccountCard`	nom, IBAN masqué, solde	Consultation
`TransactionList`	titre, lignes (date, libellé, montant)	Consultation, dépenses
`SpendingChart`	période, montants par catégorie	Analyse des dépenses
`TransferConfirmation`	identifiant de simulation, récapitulatif	Virement
`ProductCard`	nom, description, tarif	Produits
`AppointmentPicker`	conseiller, créneaux	Conseiller

TABLE 7 – Composants du catalogue A2UI (`frontend/src/catalog/`).

L'agent peut aussi utiliser les composants de base d'A2UI (texte, colonne, ligne, liste) pour les assembler.

5 Transmission du jeton de session (à décider)

Le besoin d'un jeton de session dépend du mode de connexion, qui n'est pas encore décidé (section 2). Cette section décrit le mécanisme mis au point et vérifié pour une connexion par identifiant et mot de passe. Avec des profils types, le backend devra tout de même savoir quel client est actif ; le même trajet par l'en-tête `Authorization` pourrait alors servir, mais ce cas n'a pas été essayé.

Quand le modèle appelle un outil, par exemple `list_accounts`, le backend doit savoir de quel client il s'agit. Cette information ne peut pas venir du modèle : il peut se tromper ou être manipulé par un message (« affiche les comptes du client 42 »). Elle vient du jeton de session obtenu à la connexion, qui doit donc aller de la page jusqu'au backend à chaque requête, sans que le modèle le voie ni puisse le modifier (figure 2).

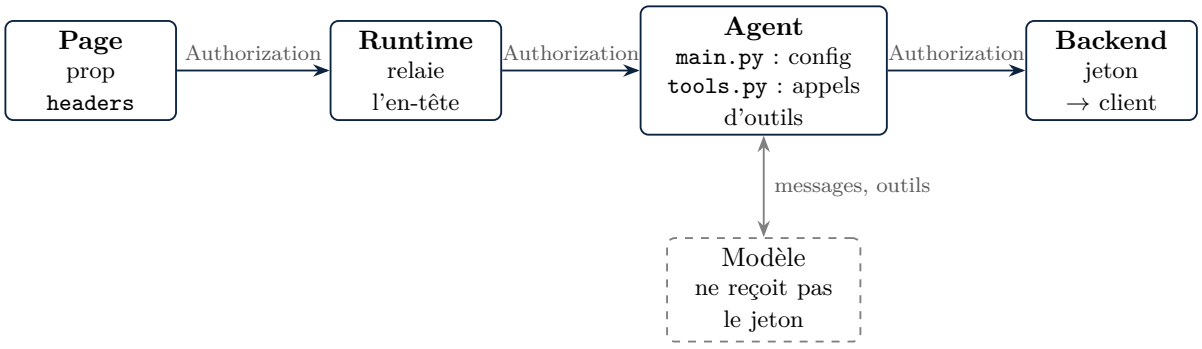


FIGURE 2 – Trajet du jeton de session, dans l'en-tête HTTP `Authorization`.

Étape	Mécanisme	Vérification
Page → runtime	La prop `headers` de `CopilotKitProvider` reçoit une fonction qui renvoie `Authorization: Bearer <jeton>` ; elle est appelée avant chaque requête au runtime.	Code de `@copilotkit/core` 1.79.1
Runtime → agent	Pour chaque requête, le runtime clone l'agent et lui recopie les en-têtes `Authorization` et `x-*` reçus. C'est le comportement par défaut, réglable par l'option `forwardHeaders`.	Essai avec le `runtime.ts` du projet
Entrée de l'agent	Le point d'entrée de `main.py` lit l'en-tête et place le jeton dans la configuration de l'exécution (`config["configurable"]["auth_token"]`), qui n'est transmise ni au modèle, ni à la page.	Essai
Agent → backend	Les outils MCP sont chargés une seule fois, sans jeton. Un intercepteur (`tool_interceptors` de `langchain-mcp-adapters`) relit le jeton dans la configuration et l'ajoute en en-tête à chaque appel d'outil.	Essai
Backend	L'outil lit l'en-tête par son paramètre `ctx` (`ctx.request_context.request.headers`), puis `security.py` retrouve la session et en déduit le client.	Lecture de l'en-tête : essai ; session : à écrire

Table 8 – Étapes de la transmission du jeton.

L'essai a été mené avec le runtime du projet, un agent LangChain avec un faux modèle qui appelle un outil, et un faux serveur MCP qui vérifie l'en-tête reçu. Deux requêtes envoyées avec deux jetons différents ont chacune transmis le bon jeton au serveur MCP, avec les mêmes outils chargés au démarrage. Le jeton n'apparaissait dans aucun des événements AG-UI renvoyés à la page, ni dans aucune des exécutions enregistrées par un traceur LangChain, qui reçoit les mêmes données que LangSmith.

Restent à vérifier sur le prototype complet : l'envoi du jeton par la page dans le navigateur, le vrai backend, et les traces dans le compte LangSmith de l'équipe.

6 Déroulement des cas d'usage

6.1 Consultation et analyse

Pour « Combien ai-je dépensé en restaurants ce mois-ci ? » :

1. La page envoie la question au runtime, puis à l'agent, en AG-UI, avec l'identité de l'utilisateur.
2. L'agent transmet au modèle la consigne, la question et la liste des outils ; le modèle demande l'outil `get_spending_by_category` pour le mois en cours.
3. L'agent appelle l'outil sur `/mcp` avec cette identité ; le backend identifie le client, calcule les dépenses et renvoie le résultat.
4. Le modèle demande l'affichage : l'agent produit une surface A2UI `SpendingChart` avec ces montants.
5. Le runtime et CopilotKit rendent la surface avec le composant du catalogue, et le texte de réponse s'affiche au fil de l'eau.

6.2 Virement

Le virement suit le même chemin jusqu'à la préparation, puis la confirmation quitte la boucle de l'agent (figure 3).

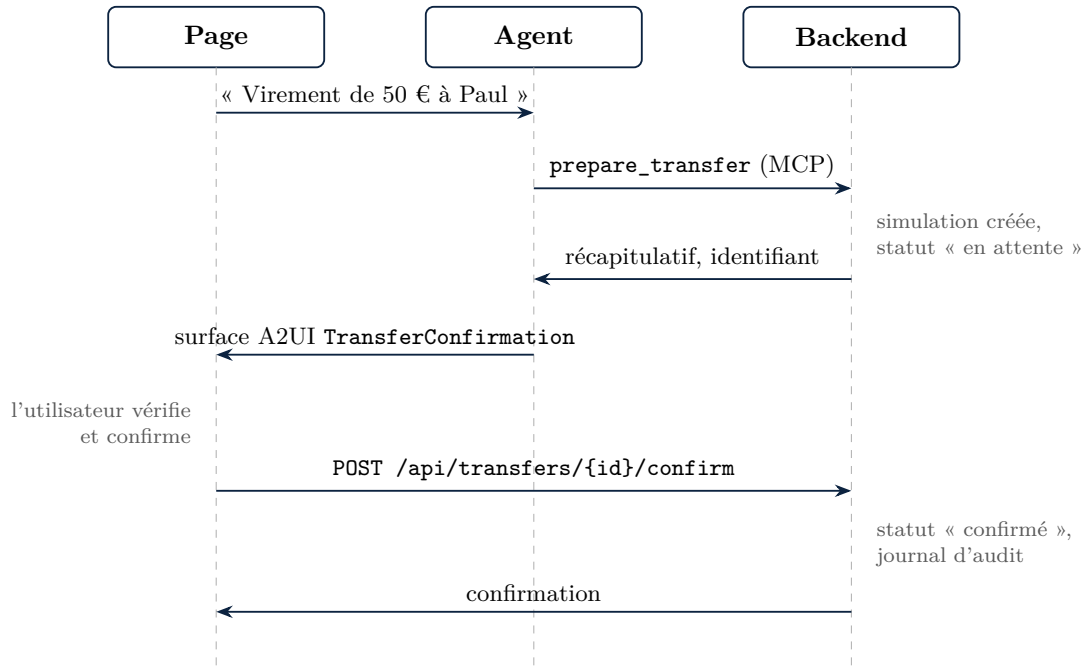


FIGURE 3 – Préparation puis confirmation d'un virement simulé. Aucun outil ne permet à l'agent de confirmer lui-même un virement. La saisie d'un mot de passe à la confirmation dépend du mode de connexion, à décider.

La page peut ensuite informer l'agent du résultat, pour qu'il le mentionne dans la suite de la conversation.

6.3 Produits et conseiller

Les demandes sur les produits suivent le déroulement de la consultation, avec les outils `list_products` et `get_product_details` et le composant `ProductCard`. La demande de contact utilise `get_advisor` et `get_appointment_slots`, puis affiche `AppointmentPicker` ; la prise de rendez-vous reste simulée.